

SYSTEM ANALYSIS DOCUMENTATION (SAD)

UMHackathon 2026

umhackathon@um.edu.my

1. Introduction:

This section introduces the Intelligent Warehouse Restock Manager, a strategic solution designed specifically for an Electrical Trading Warehouse. The system addresses the logistical complexities of managing diverse electrical components—ranging from miniature switches to heavy distribution boards—by optimizing time, cost, and storage space through AI-driven automation.

1.1. Purpose:

The system analysis document highlights the technical scope and design-related decisions behind the development of the Intelligent Warehouse Restock Manager. The key elements of the system covered are as follows:

1. **Architecture:** A robust client-server platform. The frontend is built with React for high-energy, responsive mobile interaction, while the backend utilizes a Java-based Javalin REST API to handle complex warehouse logic and AI integration.
2. **Data Flows:** Details how inventory data and worker inputs flow from the mobile interface through the backend reasoning engine, are verified against persistent SQLite databases and CSV order files, and finally return clear, actionable instructions to the warehouse worker.
3. **Model Process:** Shows the end-to-end workflow of core features: automated binning for arriving stock, real-time "Customer Order" matching (Cross-Docking), and emergency hazard reporting.
4. **Role of Reference:** This document serves as a technical reference for developers and stakeholders to understand the high-level architecture and deterministic logic ensuring zero-hallucination AI operations.

1.2. Background:

Entrepreneurs in the electrical trading sector often manage large warehouses with thousands of technical SKUs. They have found a market where the restock and organization of heavy or technical electrical stock is quite manual, leading to significant delays in product turnaround and high labor costs.

1. **Previous Version:** The existing operations are conducted via physical logs and verbal instructions. The warehouse management currently relies on the physical presence of a manager to decide where stock goes and to cross-reference customer orders manually. If workers are new or unfamiliar with specific electrical parts, they must halt operations to reach out to a manager for identification and placement instructions.

2. Changes in Major Architectural Components:

- **Deterministic AI Reasoning Layer:** Shift from manual decision-making to a "gated" reasoning layer where the NVIDIA Llama-3.3 engine is restricted by Java validation logic to prevent hallucinations.
- **Asynchronous Notification Service:** Implementation of a dedicated Telegram Bot API integration for real-time manager alerts, replacing manual verbal reporting.
- **Persistent SQL Edge Storage:** Migration from physical logs to a localized SQLite database architecture for 24/7 rapid inventory querying.

3. New Capabilities that are only introduced in this version:

- **Real-Time "Protocol Omega" (Safety Rerouting):** Users can now "block" hazardous aisles using natural language, causing the system to automatically reroute other restock tasks away from that location.
- **Automated Cross-Docking (CO Match):** The system automatically parses local CSV files to identify if arriving stock is needed for high-priority customer orders, allowing for immediate packing instead of storage.
- **Intelligent Product Identification:** Workers can describe components to the AI to receive instant technical specs and storage requirements without manager intervention.

1.3. Target Stakeholder

Stakeholders	Roles	Expectations
Entrepreneurs	Target domain audience seeking to maximize warehouse ROI and business scalability.	Expects a reduction in overhead costs, optimization of existing warehouse square footage (spatial efficiency), and the elimination of growth-limiting "managerial bottlenecks."
Warehouse Managers	Responsible for safety oversight and managing high-priority order flows.	Expects real-time visibility of inventory, immediate Telegram alerts for accidents (Protocol Omega), and automated verification of restock accuracy.
Warehouse Workers	Primary users who perform physical restock, identification, and reporting.	Expects clear, text-based instructions on their mobile devices to eliminate the need for manual space searching and minimize verbal miscommunication in noisy environments.
System Administrators	Responsible for maintaining the technical infrastructure and data integrity.	Expects a stable, persistent system with normalized database schemas (SQLite) and reliable API integration for the NVIDIA reasoning engine and Telegram bot services.

2. System Architecture & Design

2.1. High Level Architecture

2.1.1. Overview:

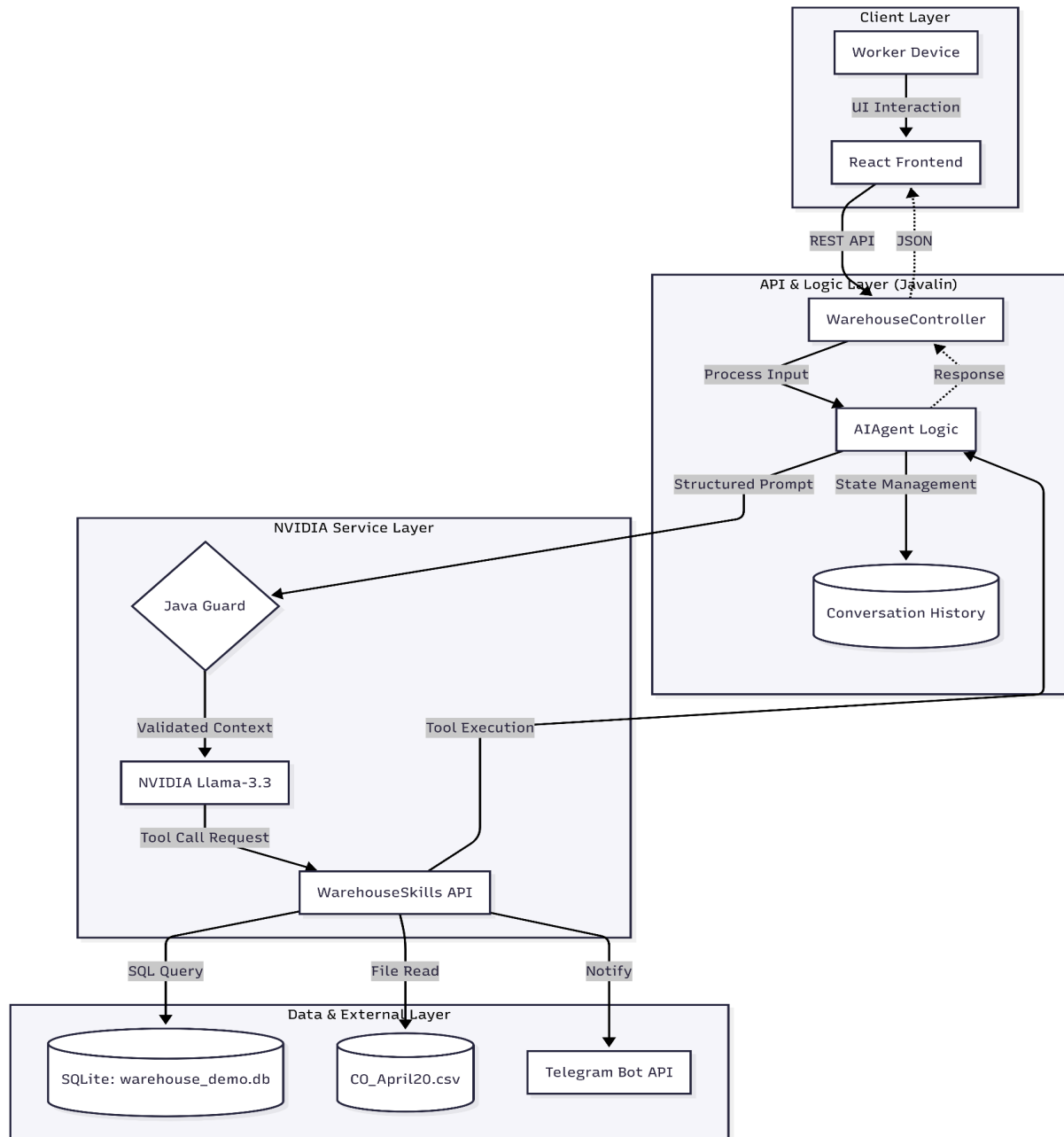
Type	Details
System	Mobile-Responsive Web Application (React)
Architecture	Client-Server with Java RESTful API and NVIDIA NIM Integration

Intelligent Warehouse Restock Manager is structured as a client-server system, with the React Frontend serving as the primary interface for Warehouse Workers. All communication is handled by a Java (Javalin) Backend API layer. The system utilizes a localized high-performance architecture: the AI Agent Service, Database Service (SQLite), and Telegram Notification Service all operate as integrated backend modules. The backend ensures persistence by hosting the **warehouse_demo.db** locally, allowing the system to operate with high availability in industrial environments.

2.1.2 LLM as Service Layer

The architecture integrates NVIDIA Llama-3.3-70b-instruct not as a generic module, but as a deterministic service layer gated by Java logic to ensure industrial reliability.

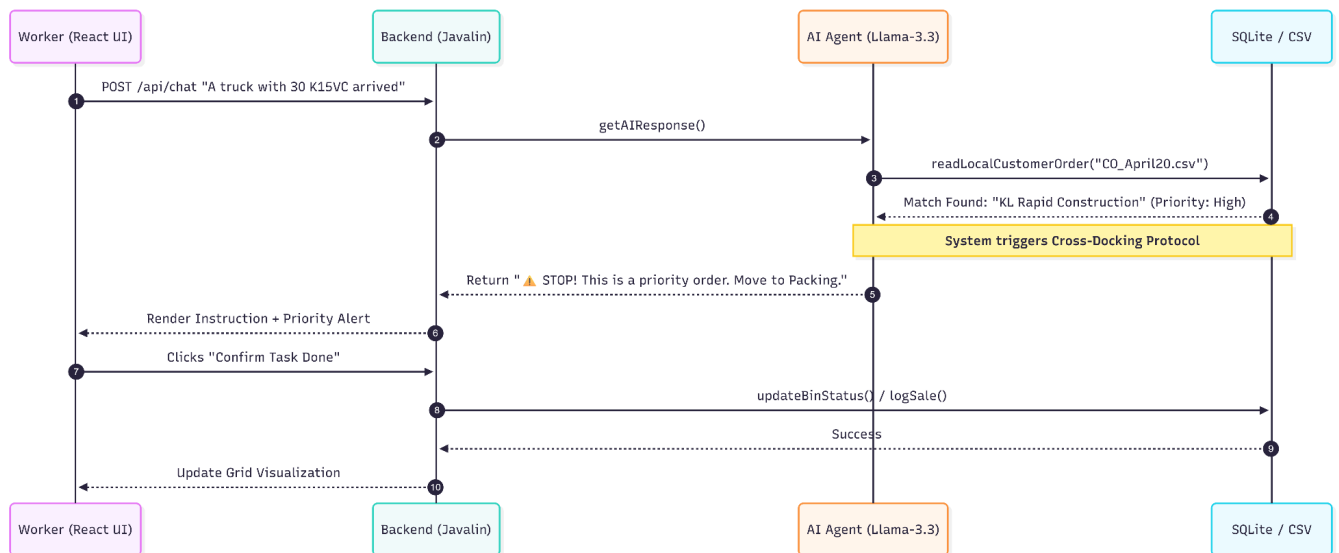
2.1.3 Dependency Diagram



- **Prompt Construction:** Uses a specialized Contextual Wrapper in **AI Agent.java**. It prepends System Instructions and appends a conversationHistory JSON array.
- **Context window:** Dynamically injects SQLite schema facts and the **CO_April20.csv** file for live grounding.
- **Parsing & Control Flow:** Utilizes a Function-Calling Loop. The Java backend intercepts **tool_calls**, executes SQL logic, and returns results to the LLM.

- **Token Enforcement:** Implements a rolling window buffer to discard old turns and prevent token overflow.
- **Major API calls:** React calls `/api/chat`. Backend calls NVIDIA NIM API and Telegram Bot API.

2.2.2. Sequence Diagram



2.2. Technological Stack

2.2.1. Frontend

- ❖ **React** – Component-based architecture for mobile responsiveness

2.2.2. Backend

- ❖ **Java (Javalin)** – RESTful API to manage warehouse micro-logic and NVIDIA integration

2.2.3. Database

- ❖ **SQLite** – Relational database for local persistence of Bins, Products, and Sales

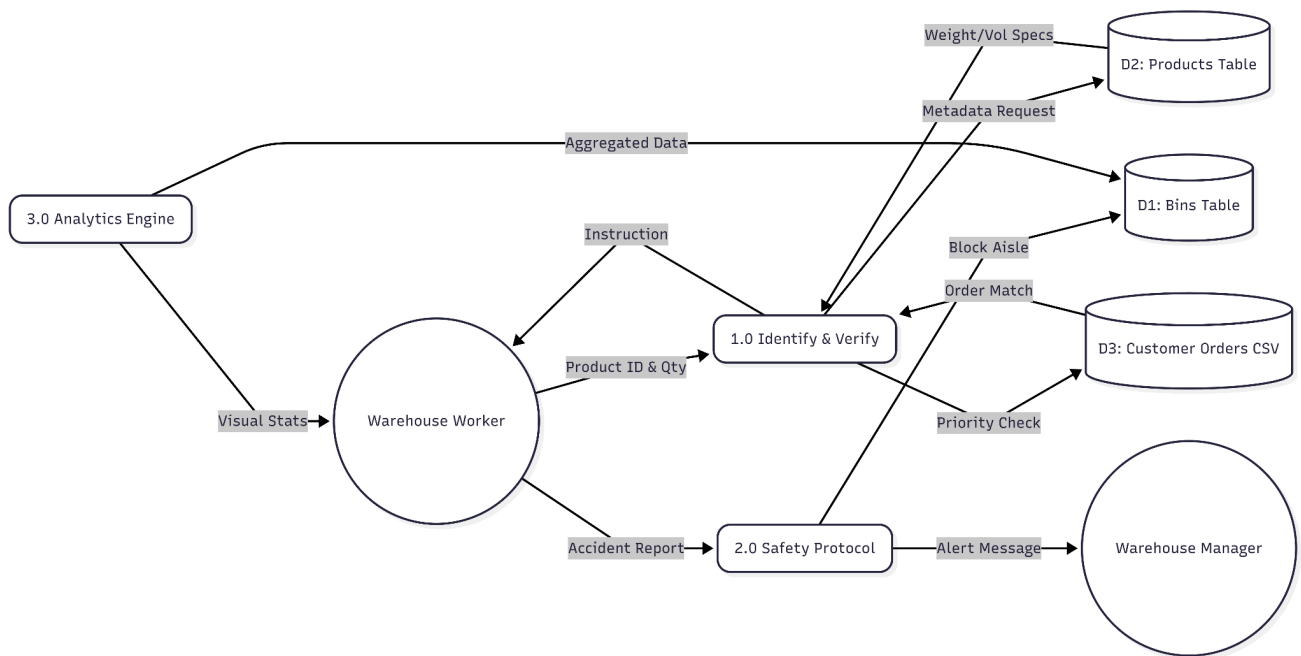
2.2.4. Cloud/Deployment

- ❖ **NVIDIA NIM** (LLM Inference) and **Telegram API** (Manager Notifications)

2.3. Key Data Flows

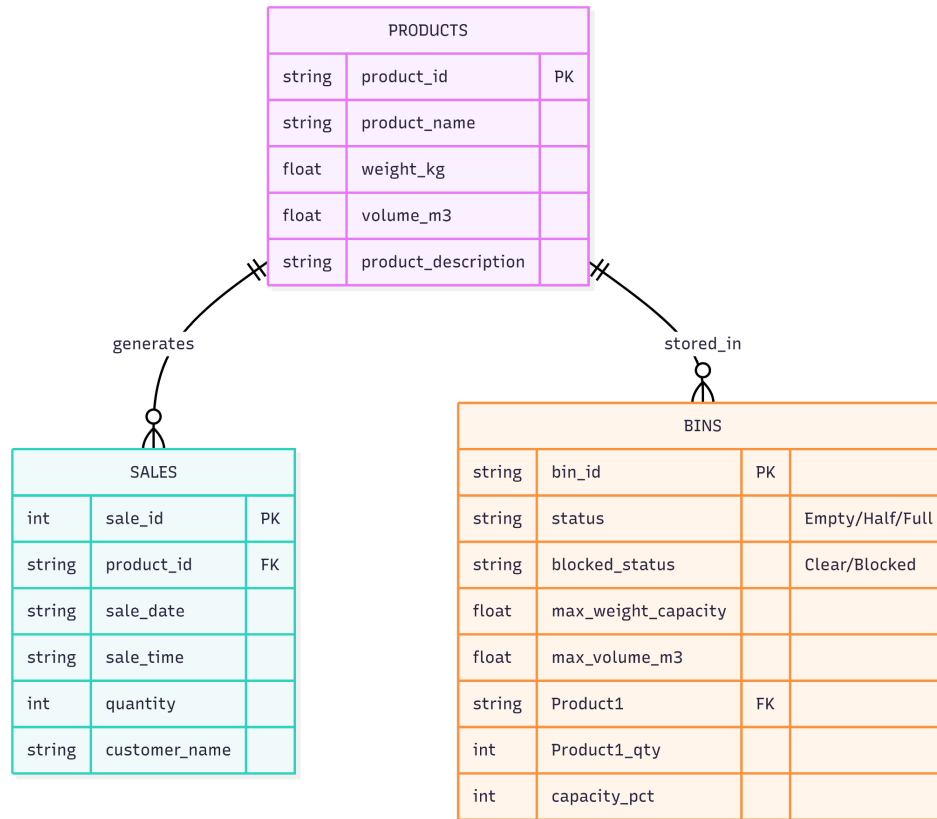
2.3.1. This section illustrates how data moves through the Intelligent Warehouse Restock Manager, transitioning from unstructured natural language to structured SQL commands and asynchronous safety alerts.

2.3.1.1. Data Flow Diagram (DFD)



- **Illustrate data movement:** Data enters as natural language. AI Agent transforms input into SQL queries for Products and CSV data.
- **Safety Integration:** Hazard reports update the Bins Table and asynchronously trigger the Telegram API.

2.3.2. Normalized Database Schema



- **Data Integrity:** The Sales table uses **product_id** as a Foreign Key to enable Sales Velocity analytics.
- **Constraint Management:** Bins table enforces physical weight/volume limits defined in the Products table.

3. Functional Requirements & Scope

This section highlights the core boundaries of the projects. That enables them to focus on a high-impact MVP to showcase the technical feasibility of the project.

3.1. Minimum Viable Product:

3.1.1. The Intelligent Warehouse Restock Manager is developed as a high-impact MVP designed to demonstrate the technical feasibility of integrating Generative AI into industrial warehouse operations. The primary focus of this MVP is to streamline the transition from manual, verbal restock instructions to a deterministic, AI-guided workflow. By prioritizing three core pillars—Intelligent Inventory Placement, Automated Cross-Docking, and Real-Time Safety Protocols—this version provides a scalable solution for electrical trading entrepreneurs to reduce labor overhead and operational downtime.

The following table outlines the core features:

#	Features	Description
1	Intelligent Restock Assistant	Workers describe arrivals in natural language. The AI identifies product specs from SQLite and suggests optimal bin placement based on volume and weight.
2	Automated Cross-Docking (CO Match)	The system intercepts incoming stock by parsing the CO_April20.csv file. If a high-priority customer match is found, it redirects the worker to "Packing."
3	Protocol Omega (Safety Rerouting)	Workers report hazards via chat. The system updates the database to block that location and reroutes other tasks to safe aisles.
4	Managerial Alert System	An integrated Telegram Bot API service that sends instant notifications to managers when safety protocols are triggered.
5	Live Operational Dashboard	Visualizes Sales Velocity and Bin Capacity using Recharts for real-time warehouse health monitoring.

3.2. Non-Functional Requirements (NFRs)

Beyond core functionalities, the Intelligent Warehouse Restock Manager is built to adhere to strict industrial standards to ensure it is viable for daily warehouse operations. These Non-Functional Requirements (NFRs) focus on the system's stability, the precision of the AI reasoning, and the responsiveness of the interface. Given the high-stakes nature of electrical trading inventory, the implementation ensures that the system remains a reliable single source of truth for all stock movements and safety protocols.

The following table highlights the system qualities that are not features but are crucial for reliability and system improvement:

Quality	Requirements	Implementation
Reliability	Inventory updates and safety blocks must be persistent and survive system restarts.	Uses a localized SQLite database (warehouse_demo.db) with transactional integrity for all bin and product updates.
Maintainability	The system must separate high-level AI reasoning from low-level database actions.	Implemented via a Java (Javalin) REST API where AI Agent handles logic and WarehouseSkills handles data execution.
Token Latency	AI responses must be fast enough for active warehouse floor use (average response < 3s).	Utilizes the NVIDIA Llama-3.3 NIM for high-speed inference and keeps a limited rolling window for conversation history.
Cost Efficiency	Prevent excessive API costs from long, repetitive queries or redundant context injection.	Prompts are grounded in a fixed local schema; frequently accessed product metadata is pulled from SQLite rather than generating it via LLM.
Safety	Hazardous areas must be strictly inaccessible to the automated bin-finding logic.	The blockBinsByLocation method sets a hard blocked_status flag in the DB, which the AI is programmed to skip during its search.

3.3. Out of Scope / Future Enhancements

While the current version of the Intelligent Warehouse Restock Manager establishes a solid foundation for AI-driven inventory management, several advanced features have been identified for the "Grand Vision" of the platform. These enhancements focus on increasing automation and predictive intelligence to further optimize the supply chain. Although these features are not part of the initial MVP, they represent the next phase of the system's evolution, as outlined in the list below:

- a. **Autonomous Mobile Robots (AMR) Integration:** Connecting the API directly to automated forklifts for physical stock movement.
- b. **Computer Vision (CV) Scanning:** Utilizing real-time camera feeds to identify electrical components via image recognition.
- c. **Predictive Restock Forecasting:** Using historical sales data to automatically generate purchase orders for low-stock items.
- d. **Multi-Warehouse Synchronization:** Cloud-based scaling to manage inventory across multiple geographical locations.

4. Monitor, Evaluation, Assumptions & Dependencies

4.1. Technical Evaluation:

4.1.1. Grayscale Rollout & A/B Testing:

The system follows a phased deployment strategy. Initially, the **AI-Driven Bin Suggestion** feature is released to a specific warehouse zone (5% of active worker sessions) to monitor database write speeds and AI response accuracy. Once validated, A/B testing is conducted between the "LLM-Only" bin assignment logic and the "LLM + Java Heuristics" logic to determine which variant results in lower physical travel time for workers.

4.1.2. Emergency Rollback (ER) & Golden Release:

In the event of system-wide anomalies (e.g., recursive tool-calling loops or SQLite database locks), an **ER** is triggered. The deployment pipeline is configured to automatically revert to the **Golden Release**—a stable version of the Java backend that bypasses the AI layer and utilizes a basic FIFO (First-In, First-Out) bin assignment logic to maintain warehouse operations.

4.1.3. Priority Matrix:

- 4.1.3.1. To ensure the **Intelligent Warehouse Restock Manager** remains reliable in a high-stakes industrial environment, a clear Priority Matrix has been established. This matrix categorizes potential technical failures based on their impact on warehouse operations and safety. By defining specific trigger conditions and automated actions, the system can maintain a "Fail-Safe" state—ensuring that critical warehouse tasks continue even if advanced AI features experience latency or external service interruptions.

The following table defines the priority levels, the conditions that trigger them, and the corresponding technical actions taken by the system:

Priority	Level	Condition (Trigger)	Action (Resolution)
P1	Critical	NVIDIA NIM API latency > 10s OR warehouse_demo.db connection failure.	Emergency Rollback: System automatically switches to "Local Heuristic Mode," bypassing AI for basic bin assignment.
P2	High	Telegram Bot fails to deliver "Protocol Omega" safety alerts within 5 seconds of an accident report.	Failover Alert: Trigger local UI-based pop-ups on all worker devices and force-log the hazard to the central dashboard.
P3	Medium	CO_April20.csv file is missing, unparseable, or contains corrupted SKU IDs.	Graceful Degradation: Skip Cross-Docking checks; default to standard Restock logic and notify the Manager.
P4	Low	Cosmetic glitches in Recharts visualization or non-critical React UI styling bugs.	Scheduled Maintenance: Log the issue for the next deployment cycle without interrupting active warehouse tasks.

4.2. Monitoring:

The monitoring strategy for the **Intelligent Warehouse Restock Manager** is designed to provide real-time oversight of the system's "Vital Signs." The objective is to maintain a high-availability environment where the AI reasoning remains deterministic and the safety alert protocols (Protocol Omega) are fail-safe.

4.2.1. System Health & Trigger Planning

This section details the specific planning for damage detection and the automated triggers that execute when the system's health is compromised.

1. AI Logic & Reasoning Monitoring (AIAgent . java):

- **Planning:** The system monitors the **NVIDIA Llama-3.3 NIM API** availability. Every request is wrapped in a latency-check timer to ensure the warehouse worker isn't left waiting on the floor.
- **Trigger:** Detection of a 500 Internal Server Error from the NVIDIA endpoint or a response time exceeding **10 seconds**.
- **Action:** The system triggers a **P1 Critical State**. The backend immediately notifies the React UI to display a "**Local Heuristic Mode**" banner, allowing workers to continue bin assignments using pre-set Java logic instead of AI reasoning.

2. Data Integrity & Consistency Watch (warehouse_demo.db):

- **Planning:** During every restock transaction, the system monitors for **SQLite write conflicts** or database locks.
- **Trigger:** Detection of a **SQLITE_BUSY** or **SQLITE_CORRUPT** error code during an attempt to update a Bin's **capacity_pct**.
- **Action:** To prevent "Orphaned Stock" (where physical items are on shelves but not in the DB), the system triggers a **Data-Lock Protocol**. The "Confirm" button is disabled in the frontend, and the current transaction is rolled back to the last known stable state.

3. Safety Communication Redundancy (Protocol Omega):

- **Planning:** The system tracks the delivery success of the **Telegram Bot API** for every reported accident or aisle blockage.
- **Trigger:** If the Telegram server fails to return a **200 OK** acknowledgment within **3 seconds**.
- **Action:** The system identifies a "Safety Gap" and escalates the alert. It bypasses the external bot and triggers a **High-Priority UI Broadcast** across all active Manager Dashboards in the warehouse to ensure immediate human intervention.

4. Automated Recovery & Self-Healing:

- **Planning:** Once a monitoring probe detects that services (API and Database) have returned to a "Green" status.
- **Trigger:** Three consecutive successful "Heartbeat" pings from the Java backend to the external services.
- **Action:** Before resuming full AI operations, the system performs a **State Validation Check**, cross-referencing the **Products** table and the CO_April20.csv to ensure no data was lost or desynchronized during the downtime.

4.3. Assumptions:

This section states the key operational and environmental conditions assumed to be valid during the development and deployment of the Intelligent Warehouse Restock Manager.

- ❖ **Network Stability:** It is assumed that warehouse workers have stable internal Wi-Fi or mobile data connectivity to maintain a persistent connection between the React frontend and the Javalin backend server.
- ❖ **Hardware Availability:** We assume that the warehouse management provides workers with handheld mobile devices or tablets capable of running a modern web browser to access the system's interface.
- ❖ **Data Accuracy (Initial State):** It is assumed that the initial product dimensions (weight and volume) stored in the **Products** table are accurate, as these form the basis for all AI-driven bin placement calculations.
- ❖ **Manager Accessibility:** We assume that warehouse managers have active Telegram accounts and have successfully initiated a chat with the **Protocol Omega Bot** to receive emergency notifications.
- ❖ **Server Environment:** It is assumed that the warehouse hosts a dedicated local server or a stable cloud instance to run the Java Virtual Machine (JVM) and store the **warehouse_demo.db** file.
- ❖ **Standardized Labeling:** We assume that electrical components arriving at the warehouse have clear SKU IDs or descriptions that match the identification logic used by the NVIDIA reasoning engine.

4.4. External Dependencies:

The following external tools and services are critical to the system's functionality. Each dependency is managed with a specific mitigation strategy to ensure warehouse operations remain resilient.

Tools	Purpose	Risks
NVIDIA NIM API (Llama-3.3)	Serves as the core LLM reasoning engine. It handles natural language parsing, multi-step bin assignment logic, and generating structured JSON tool calls.	High – If the API reaches rate limits or experiences downtime, the "Intelligent" aspect of the restock fails. Mitigation: Implement a local heuristic fallback mode using standard Java logic.
Telegram Bot API	Facilitates the "Protocol Omega" safety system by sending instant, asynchronous hazard alerts to the management team.	Medium – If the Telegram service is throttled or the bot token expires, emergency alerts may not reach managers' phones. Mitigation: Redundant logging to the local Web Dashboard.
SQLite JDBC	Provides the persistent storage layer for the "Digital Twin" of the warehouse, managing all product metadata, bin occupancy, and sales data.	Low – As a local file-based database, risk is minimal. However, disk space issues could cause write failures. Mitigation: Automated 24-hour database backups and integrity checksums.
React / Recharts	Used for the frontend interface and data visualization of warehouse throughput and sales velocity.	Low – Dependency on browser compatibility and npm package stability. Mitigation: Use of stable LTS versions and mobile-responsive CSS for handheld warehouse devices.

5. Project Management & Team Contributions

The development of the Intelligent Warehouse Restock Manager was executed over a 10-day intensive sprint, focusing on delivering a functional AI-integrated MVP for the electrical trading sector.

5.1. Project Timeline:

Timeline	Phase	Justification
Day 1-2	Project Planning & Requirements Identification	Defined the core logic for the "Protocol Omega" safety system and mapped out the SQL schema for electrical component metadata.
Day 3-5	Backend Development & AI Integration	Developed the AIAgent.java and integrated the NVIDIA Llama-3.3 NIM API to handle natural language restock instructions and tool-calling logic.
Day 6-8	Frontend Development & Data Visualization	Built the mobile-responsive React interface and implemented Recharts to visualize warehouse throughput and sales velocity in real-time.
Day 9-10	System Integration & Protocol Omega Testing	Conducted end-to-end testing of the Telegram Bot alert escalation and verified the local SQLite fallback logic during simulated API downtime.

5.2. Team Members Role:

❖ Teoh Jun Hong – Role: Team Leader & AI Architect

- Designed the high-level system architecture and crafted the prompt engineering strategy for the **NVIDIA Llama-3.3** reasoning layer.
- Lead the logic development for the **AIAgent.java** to ensure deterministic AI outputs.

❖ Isaac Toh – Role: Backend Lead – Java API & Schema Design

- Developed the **Javalin REST API** and managed the transactional integrity of the **SQLite** database.
- Engineered the **Telegram Bot API** integration for the "Protocol Omega" safety alert system.

❖ **Leow Zhen Xun – Role: Frontend Developer – Dashboards & UI**

- Designed and implemented the Manager’s analytics dashboard using **React**.
- Integrated **Recharts** to provide real-time visualization of warehouse sales velocity and bin occupancy.

❖ **Chong Ming Zhe – Role: Frontend Developer – Mobile Chat Interface**

- Developed the mobile-responsive worker interface to ensure accessibility on handheld devices.
- Optimized the front-end state management for seamless natural language interaction with the backend AI.

5.3. Recommendations:

To ensure the long-term **scalability** and **reliability** of the system, the following technical enhancements are recommended:

- **Redis Caching for SKU Metadata:** Implement a **Redis Cache** layer for frequently accessed electrical product specs. This would reduce the number of direct hits to the SQLite database during high-traffic shifts, significantly lowering latency.
- **Distributed Rate Limiting:** For high-volume warehouses, implement **Bucket Token rate limiting** on the API layer to manage NVIDIA NIM token consumption and prevent cost overruns.
- **Horizontal Scaling with Load Balancing:** As the warehouse scales to multiple locations, the Java backend should be containerized using **Docker** and managed via **Kubernetes**, using an **Nginx load balancer** to distribute traffic across regional instances.
- **Local Vector DB for Product Matching:** Transition from simple SQL queries to a **Vector Database (like Milvus or Chroma)** for SKU identification. This would allow for "fuzzy matching" of product descriptions even when workers use inconsistent terminology.